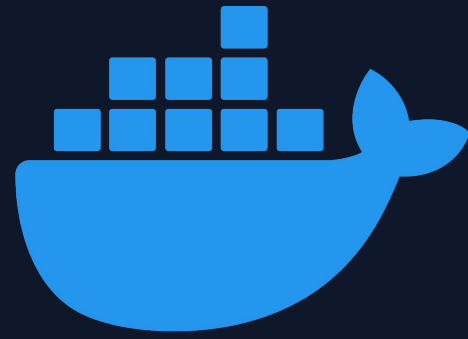




docker®

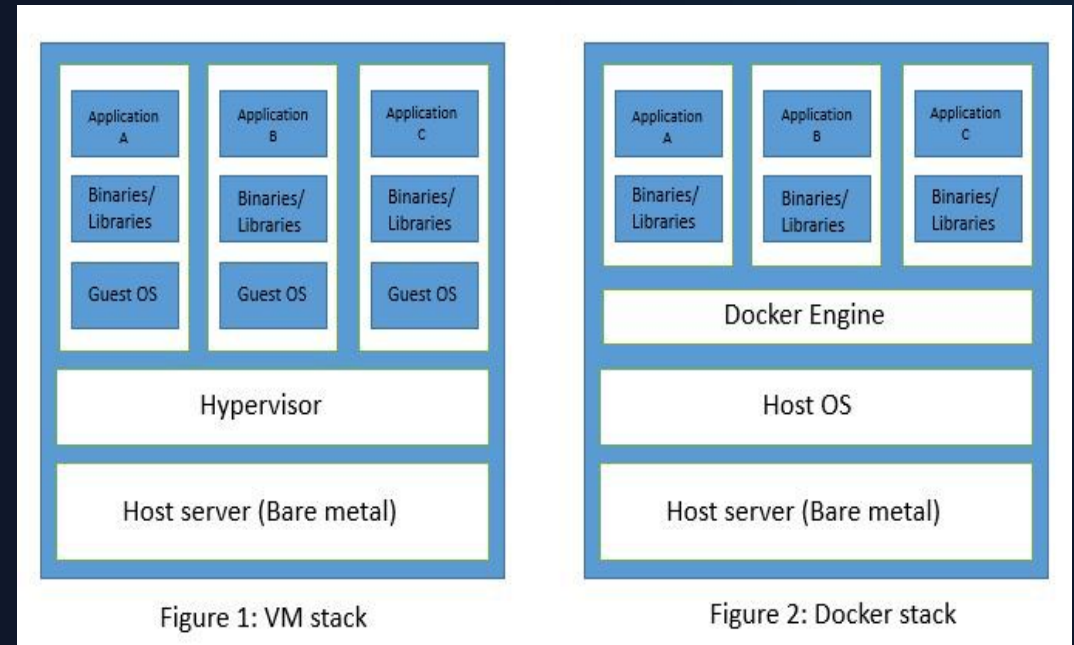
Docker & Modern Containerization

From application packaging & isolation mechanics to multi-container
development workflows.

Mihut, Luca-Adrian

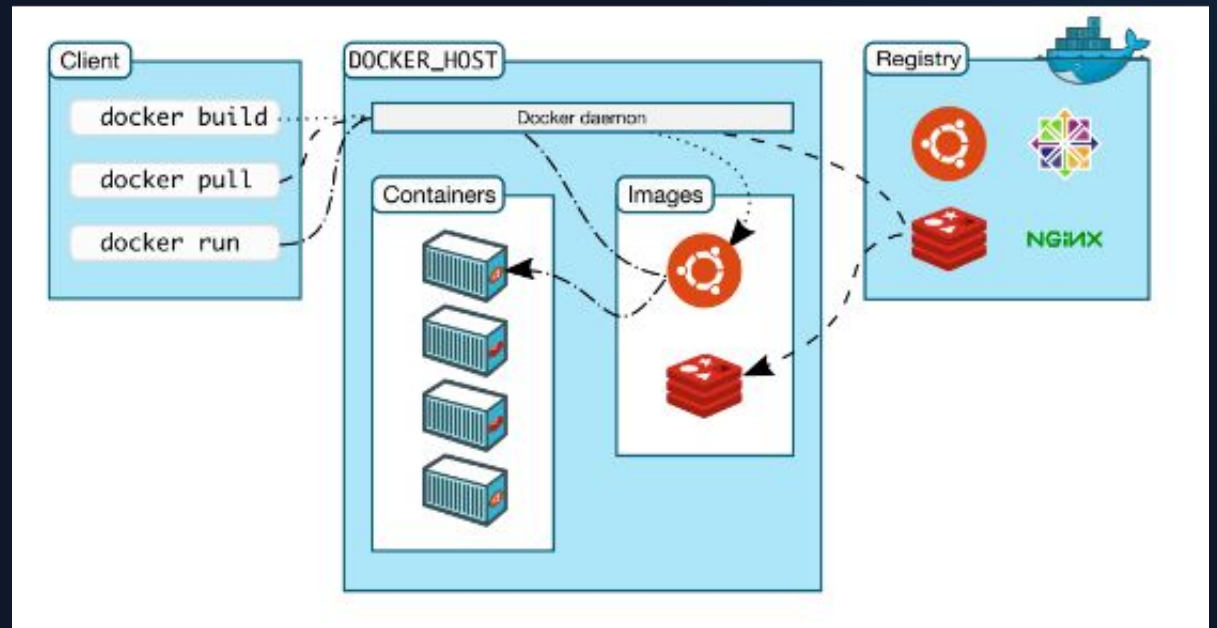
VMs vs. Modern Containers

- ⚠️ **The Matrix of Doom:** Traditional deployments suffer from dependency drift across OS environments.
- 📦 **Virtual Machines:** Include full Guest OS, hypervisor overhead, slow startup times, and gigabyte-scale footprints.
- 📦 **Containers:** Share the host OS kernel while isolating processes via Linux namespaces and cgroups.
- ⚡ **Key Benefits:** Sub-second startup times, minimal resource utilization, and predictable runtime behavior.



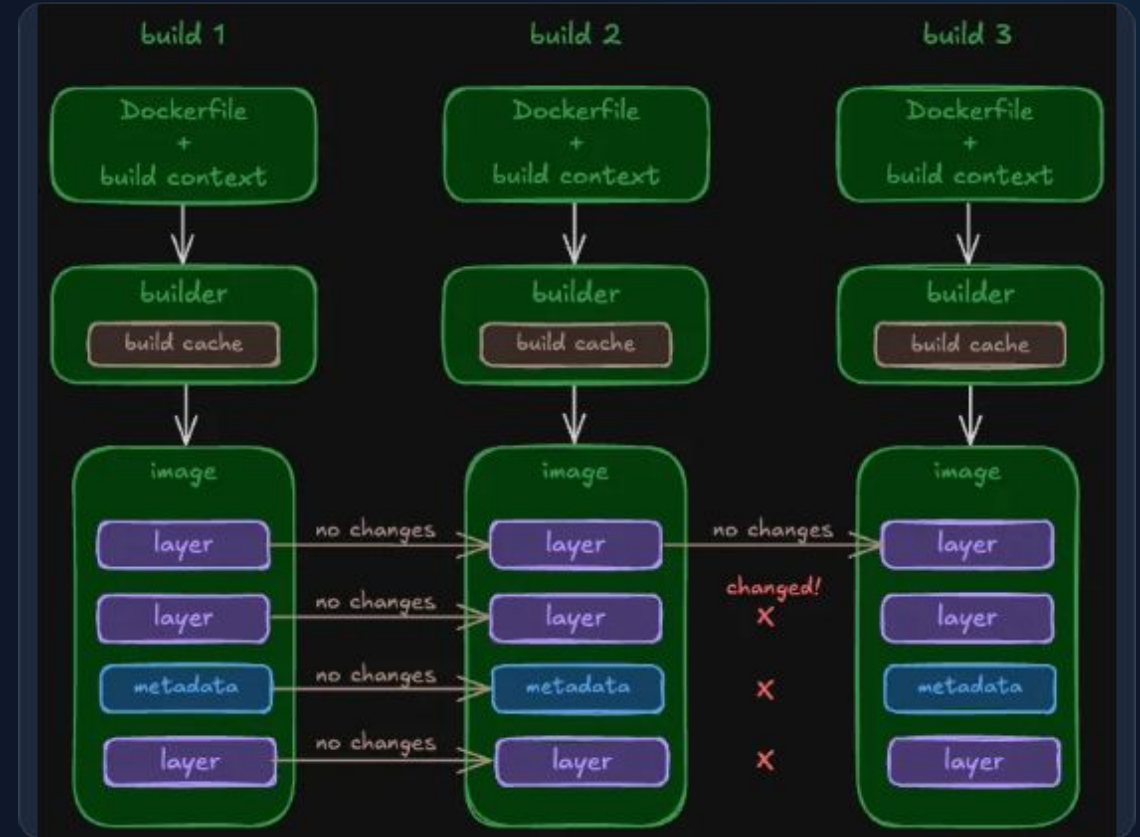
Docker Core Architecture

-  **Client-Server Model:** The Docker CLI communicates with the background dockerd daemon via REST API.
-  **Daemon Execution:** Manages images, containers, storage volumes, and virtual network interfaces.
-  **Docker Registries:** Central stores (e.g., Docker Hub, ECR) for pulling base images and pushing tagged releases.
-  **OCI Runtime:** Uses containerd and runc under the hood for low-level process invocation.



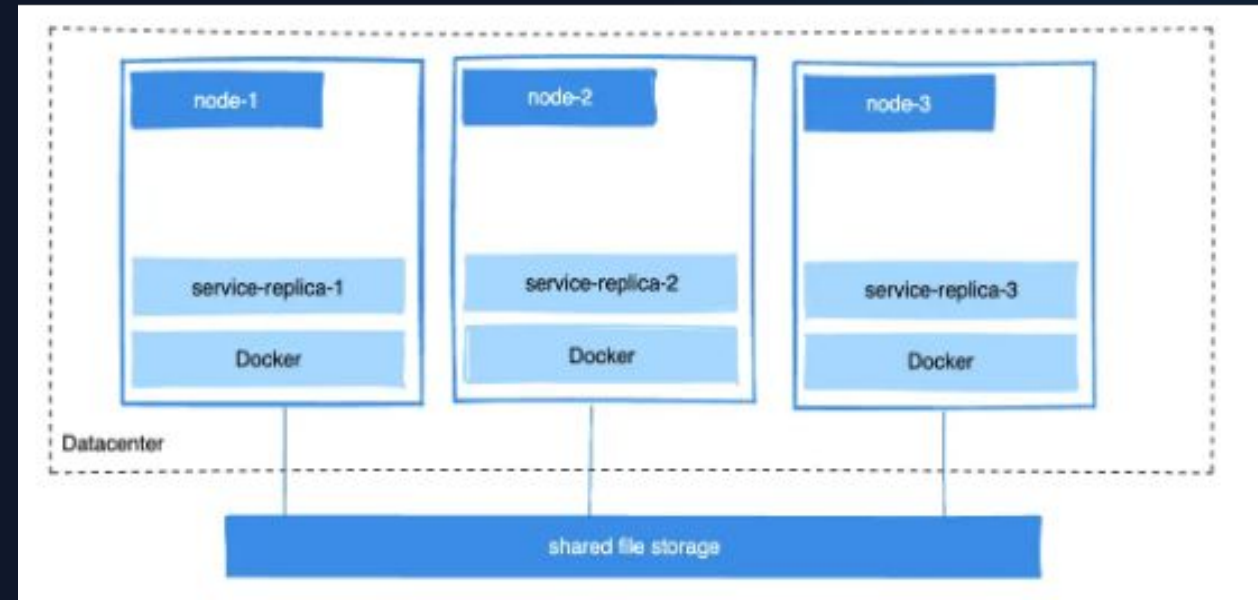
Images, Layers & Dockerfile

- 📁 **Read-Only Layers:** Every instruction in a Dockerfile creates an immutable filesystem layer.
- 📄 **Copy-on-Write (CoW):** Container runtime adds a lightweight writable layer on top of shared base layers.
- 🔗 **Multi-Stage Builds:** Separate build dependencies from production runtimes to shrink final image size.
- 🏗️ **Build Cache Optimization:** Order Dockerfile commands from least to most frequently changing to maximize cache hits.



Storage, Networks & Compose

-  **Data Persistence:** Use Docker Named Volumes for persistent database storage outside container lifecycles.
-  **Container Networking:** User-defined bridge networks enable automatic DNS resolution between service names.
-  **Docker Compose:** Declarative YAML specification (docker-compose.yml) to run multi-container applications locally.
-  **Environment Drivers:** Easily configure environment variables, volume mounts, and published ports.



Essential Docker CLI Commands

Manage the lifecycle of your containers, images, and networks with standard terminal actions.

1. Container Lifecycle

Create, start, stop, and inspect active containers. Use `-d` to run in background mode detached from terminal.

2. Image Management

Pull images from Docker Hub registries, list locally cached options, and prune unused assets to save disk space.

3. System & Diagnostics

View container log streams, inspect configuration details, and monitor CPU/memory performance in real time.

```
# 1. Spin up an Nginx container in background
```

```
docker run -d -p 8080:80 --name webserver nginx
```

```
# 2. View running containers
```

```
docker ps -a
```

```
# 3. Stream webserver runtime stdout logs
```

```
docker logs -f webserver
```

```
# 4. Stop and safely destroy the container
```

```
docker stop webserver
```

```
docker rm webserver
```

```
# 5. Retrieve official Redis database image
```

```
docker pull redis:7-alpine
```

Defining Multi-Container Apps with Compose

Docker Compose simplifies multi-container orchestrations by shifting command-line configurations into a single declarative file.

- **Declarative Spec:** Define services, networks, and volumes in a single YAML file instead of long bash commands.
- **One Command:** Spin up, configure, and connect your entire stack instantly using `docker compose up`.
- **Isolated Environments:** Easily run local development, staging, or testing environments in perfect isolation.

```
# docker-compose.yml

services:
  web:
    image: nginx:alpine
    ports:
      - "80:80"
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Single-Host Limits & Next Steps

While Docker excels at running applications on a single machine, enterprise scale requires automated scheduling, multi-node routing, and self-healing resilience across a cluster.

Questions & Discussion

Thank you! Let's transition to container orchestration at scale.

```
$ docker run -it docker-fundamentals bash
```

Image Sources



https://miro.medium.com/1*1cMLWXgaQxci5ePWXUMrRQ.jpeg

Source: blog.gopenai.com



https://miro.medium.com/1*GoZ56yZNpG_VnGGvqhYICQ.png

Source: medium.com



<https://depot.dev/images/docker-cache-across-builds.webp>

Source: depot.dev



<https://docs.docker.com/engine/storage/images/volumes-shared-storage.webp>

Source: docs.docker.com